Halo ABCI++ 합의 플로우 분석

개요

Halo는 CometBFT의 ABCI++ 인터페이스를 구현하여 고급 합의 기능을 제공합니다. 이 문서는 `PrepareProposal`, `ProcessProposal`, `FinalizeBlock` 함수와 CometBFT-EVM 엔진 상호작용 과정을 상세히 분석합니다.

ABCI++ 인터페이스 개요

ABCI++는 기존 ABCI에 다음 기능들을 추가합니다:

- **Vote Extensions**: 검증자가 투표에 추가 데이터를 포함
- **PrepareProposal**: 제안자가 블록 제안을 준비
- **ProcessProposal**: 검증자가 제안된 블록을 검증
- **ExtendVote/VerifyVoteExtension**: 투표 확장 및 검증

1. PrepareProposal 플로우

1.1 EVM 엔진의 PrepareProposal 구현

```
func (k *Keeper) PrepareProposal(ctx sdk.Context, req *abci.RequestPrepareProposal,
*abci.ResponsePrepareProposal, error,
) {
// 10초 타임아웃 설정 - 빈 블록 제안을 방지
```

```go
timeoutCtx, timeoutCancel := context.WithTimeout(ctx.Context(), prepareTimeout)
defer timeoutCancel()
ctx = ctx.WithContext(timeoutCtx)

// 첫 번째 블록은 가스 미터 이슈로 인해 빈 블록으로 생성
if req.Height == 1 {
log.Warn(ctx, "Creating empty initial block due to gas issue", nil)
return &abci.ResponsePrepareProposal{}, nil
}

// 기존 트랜잭션이 있으면 에러 (EVM 페이로드만 허용)
if len(req.Txs) > 0 {
return nil, errors.New("unexpected transactions in proposal")
}
}
```

1.2 EVM 페이로드 생성 과정

최적화된 페이로드 사용 또는 새 페이로드 생성

```go
// 최적화된 페이로드 확인
payloadID, height, triggeredAt := k.getOptimisticPayload()
if reqHeight != height {
// 새 페이로드 생성 (네트워크 에러 시 재시도)
```

```
err := retryForever(ctx, func(ctx context.Context) (bool, error) {
fcr, err := k.startBuild(ctx, appHash, req.Time, reqHeight)
if err != nil {
log.Warn(ctx, "Preparing proposal failed: build new evm payload (will retry)", e
return false, nil // 재시도
}
// 페이로드 상태 검증
if isSyncing(fcr.PayloadStatus) {
return false, errors.New("evm unexpectedly syncing")
}
if invalid, err := isInvalid(fcr.PayloadStatus); invalid {
return false, errors.Wrap(err, "proposed invalid payload")
}
payloadID = *fcr.PayloadID
return true, nil // 완료
})
}
```

EVM 엔진과의 상호작용

```
func (k *Keeper) startBuild(ctx context.Context, appHash common.Hash, timestamp
// 현재 실행 헤드 가져오기
head, err := k.getExecutionHead(ctx)
if err != nil {
```

```
return engine.ForkChoiceResponse{}, errors.Wrap(err, "latest execution block")
}

// 타임스탬프 조정 (실행 블록은 고유한 초 단위 타임스탬프 필요)
ts, err := umath.ToUint64(timestamp.Unix())
if  ts <= head.GetBlockTime() {
ts = head.GetBlockTime() + 1
}

// CometBFT는 즉시 최종성을 가지므로 head/safe/finalized는 모두 최신 높이
fcs := engine.ForkchoiceStateV1{
HeadBlockHash: headHash,
SafeBlockHash: headHash,
FinalizedBlockHash: headHash,
}

// 페이로드 속성 설정
attrs := &engine.PayloadAttributes{
Timestamp: ts,
Random: headHash, // 헤드 블록 해시를 randao로 사용
SuggestedFeeRecipient: k.feeRecProvider.LocalFeeRecipient(),
Withdrawals: withdrawals,
BeaconRoot: &appHash, // 앱 해시를 비콘 루트로 사용
```

```
}

// Geth 엔진에 포크초이스 업데이트 요청
resp, err := k.engineCl.ForkchoiceUpdatedV3(ctx, fcs, attrs)
return  resp, err
}
```

1.3 투표 확장 메시지 수집

```
// 이전 높이의 투표 확장에서 메시지 수집
prevHeight := umath.SubtractOrZero(reqHeight, 1)
voteMsgs, err := k.voteProvider.PrepareVotes(ctx, req.LocalLastCommit, prevHeigh
if  err != nil {
return  nil, errors.Wrap(err, "prepare votes")
}

// 실행 페이로드 메시지 구성
payloadMsg := &types.MsgExecutionPayload{
Authority: authtypes.NewModuleAddress(types.ModuleName).String(),
BlobCommitments: blobCommitments,
ExecutionPayloadDeneb: payloadProto,
}
```

```go
// 투표 메시지와 페이로드 메시지를 단일 트랜잭션으로 결합
b := k.txConfig.NewTxBuilder()
if err := b.SetMsgs(append(voteMsgs, payloadMsg)...); err != nil {
return nil, errors.Wrap(err, "set tx builder msgs")
}
```

2. ProcessProposal 플로우

2.1 제안 검증 과정

```go
func makeProcessProposalHandler(router *baseapp.MsgServiceRouter, txConfig clie
return func(ctx sdk.Context, req *abci.RequestProcessProposal) (*abci.ResponseP
// 10분 타임아웃 설정
timeoutCtx, timeoutCancel := context.WithTimeout(ctx.Context(), processTimeout)
defer timeoutCancel()

// 첫 번째 제안은 반드시 비어있어야 함
if req.Height == 1 {
if len(req.Txs) > 0 {
return rejectProposal(ctx, errors.New("first proposal not empty"))
}
return &abci.ResponseProcessProposal{Status: abci.ResponseProcessProposal_ACCEPT
```

```
}

// 쿼럼 투표 확인
var totalPower, votedPower int64
for _, vote := range req.ProposedLastCommit.Votes {
totalPower += vote.Validator.Power
if vote.BlockIdFlag == cmttypes.BlockIDFlagCommit {
votedPower += vote.Validator.Power
}
}
if totalPower*2/3 >= votedPower {
return rejectProposal(ctx, errors.New("proposed doesn't include quorum votes ex
}
}
}
```

2.2 메시지 타입 검증

```
// 허용된 메시지 타입과 개수 제한
allowedMsgCounts := map[string]int{
sdk.MsgTypeURL(&etypes.MsgExecutionPayload{}): 1, // EVM 실행 페이로드 1개만 허용
sdk.MsgTypeURL(&atypes.MsgAddVotes{}): 1, // 증명 모듈 투표 추가 1개만 허용
}
```

```go
for _, rawTX := range req.Txs {
tx, err := txConfig.TxDecoder()(rawTX)
if err != nil {
return rejectProposal(ctx, errors.Wrap(err, "decode transaction"))
}

// 트랜잭션 검증 (서명 필드가 모두 비어있어야 함)
if err := verifyTX(tx); err != nil {
return rejectProposal(ctx, errors.Wrap(err, "verify transaction"))
}

for _, msg := range tx.GetMsgs() {
typeURL := sdk.MsgTypeURL(msg)
// 메시지 타입이 예상된 것인지 확인
if i, ok := allowedMsgCounts[typeURL]; !ok {
return rejectProposal(ctx, errors.New("unexpected message type", "msg_type", ty
} else if i <= 0 {
return rejectProposal(ctx, errors.New("message type included too many times", "
}
allowedMsgCounts[typeURL]--

// 메시지 핸들러 실행
```

```
handler := router.Handler(msg)
if  handler == nil {
return  rejectProposal(ctx, errors.New("msg handler not found [BUG]", "msg_type"
}

_, err := handler(ctx, msg)
if  err != nil {
return  rejectProposal(ctx, errors.Wrap(err, "execute message"))
}
}
}
```

2.3 EVM 페이로드 검증

```
func (s proposalServer) ExecutionPayload(ctx context.Context, msg *types.MsgExec
// 제안된 페이로드 파싱 및 검증
payload, err := s.parseAndVerifyProposedPayload(ctx, msg)
if  err != nil {
return  nil, err
}

// Blob 커밋먼트에서 블록 해시 생성
```

```
blockHashes, err := blobHashes(msg.BlobCommitments)
if  err != nil {
return  nil, errors.Wrap(err, "blob commitments")
}


// EVM에 페이로드 푸시 (네트워크 에러 시 재시도)
err = retryForever(ctx, func(ctx context.Context) (bool, error) {
status, err := pushPayload(ctx, s.engineCl, payload, blockHashes)
if  err != nil {
log.Warn(ctx, "Verifying proposal failed: push new payload to evm (will retry)",
return false, nil  // 재시도
}
if  invalid, err := isInvalid(status);  invalid {
return false, errors.Wrap(err, "invalid payload, rejecting proposal")
}
if  isSyncing(status) {
log.Warn(ctx, "Can't properly verifying proposal: evm syncing", err, "payload_he
}
return true, nil  // 완료
})


return &types.ExecutionPayloadResponse{}, nil
```

```
}
```

3. Vote Extensions 시스템

3.1 ExtendVote 구현

```
func (k *Keeper) ExtendVote(ctx sdk.Context, _ *abci.RequestExtendVote) (*abci.R

// 사용 가능한 투표 가져오기

votes := k.voter.GetAvailable()


// 투표 윈도우 및 제한으로 필터링

countsByChainVer := make(map[xchain.ChainVersion]int)

duplicate := make(map[xchain.AttestHeader]bool)

var  filtered []*types.Vote

for  _, vote := range  votes {

// 투표 검증

if  err := vote.Verify();  err != nil {

return  nil, errors.Wrap(err, "verify vote")

}

// 체인 지원 확인

if  err := verifyHeaderChains(ctx, cChainID, k.portalRegistry, vote.AttestHeader

return  nil, errors.Wrap(err, "check supported chain")

}
```

```
// 중복 투표 확인 (슬래싱 방지)
if duplicate[vote.AttestHeader.ToXChain()] {
doubleSignCounter.WithLabelValues(k.voter.LocalAddress().Hex()).Inc()
log.Warn(ctx, "🚨 Skipping own duplicate slashable vote [BUG]", nil)
continue
}
duplicate[vote.AttestHeader.ToXChain()] = true

// 투표 윈도우 확인
if cmp, err := k.windowCompare(ctx, vote.AttestHeader.XChainVersion(), vote.Att
return nil, errors.Wrap(err, "windower")
} else if cmp != 0 {
continue // 윈도우 밖의 투표는 건너뛰기
}
countsByChainVer[vote.AttestHeader.XChainVersion()]++
filtered = append(filtered, vote)

// 투표 확장 제한 확인
if umath.Len(filtered) >= k.voteExtLimit {
break
}
}
```

```go
// 투표들을 프로토콜 버퍼로 마샬링
bz, err := proto.Marshal(&types.Votes{Votes: filtered})
if  err != nil {
return  nil, errors.Wrap(err, "marshal atts")
}

return &abci.ResponseExtendVote{VoteExtension: bz}, nil
}
```

3.2 VerifyVoteExtension 구현

```go
func (k *Keeper) VerifyVoteExtension(ctx sdk.Context, req *abci.RequestVerifyVot
*abci.ResponseVerifyVoteExtension, error,
) {
respAccept := &abci.ResponseVerifyVoteExtension{
Status: abci.ResponseVerifyVoteExtension_ACCEPT,
}
respReject := &abci.ResponseVerifyVoteExtension{
Status: abci.ResponseVerifyVoteExtension_REJECT,
}

height, err := umath.ToUint64(ctx.BlockHeight())
```

```
if err != nil {
return nil, err
}

// 투표 확장 파싱 및 검증
_, ok, err := k.parseAndVerifyVoteExtension(ctx, req.ValidatorAddress, req.VoteE
if err != nil {
log.Warn(ctx, "Rejecting vote extension", err, log.Hex7("validator", req.Validat
return respReject, nil
} else if !ok {
log.Warn(ctx, "Rejecting vote extension containing vote behind window", nil, log
return respReject, nil
}

return respAccept, nil
}
```

4. FinalizeBlock 플로우

4.1 ABCI 래퍼의 FinalizeBlock

```
func (l abciWrapper) FinalizeBlock(ctx context.Context, req *abci.RequestFinaliz
// 기본 애플리케이션의 FinalizeBlock 호출
```

```
resp, err := l.Application.FinalizeBlock(ctx, req)
if err != nil {
log.Error(ctx, "Finalize req failed [BUG]", err)
return resp, err
}

// 커스텀 PostFinalize 콜백 호출
header := cmtproto.Header{
Height: req.Height,
Time: req.Time,
ProposerAddress: req.ProposerAddress,
NextValidatorsHash: req.NextValidatorsHash,
AppHash: resp.AppHash, // 블록 최종화 후 앱 해시
}
sdkCtx := sdk.NewContext(l.multiStoreProvider(), header, false, nil)
if err := l.postFinalize(sdkCtx); err != nil {
log.Error(ctx, "PostFinalize callback failed [BUG]", err)
return resp, err
}

// 검증자 업데이트 로깅
for i, update := range resp.ValidatorUpdates {
log.Debug(ctx, "Validator update",
```

```
log.Hex7(fmt.Sprintf("pubkey_%d", i), update.PubKey.GetSecp256K1()),
fmt.Sprintf("power_%d", i), update.Power)
}

return resp, err
}
```

4.2 PostFinalize 최적화 빌드

```
func (k *Keeper) PostFinalize(ctx sdk.Context) error {
if !k.buildOptimistic {
return nil // 최적화 빌드가 비활성화된 경우
}

// 다음 제안자인지 확인
isNext, err := k.isNextProposer(ctx, height)
if err != nil {
log.Warn(ctx, "Next proposer failed, skipping optimistic EVM payload build", err
return nil
} else if !isNext {
return nil // 다음 제안자가 아니면 아무것도 하지 않음
}
```

```
// 최적화된 EVM 페이로드 빌드 시작
fcr, err := k.startBuild(ctx, appHash, timestamp, nextHeight)
if err != nil {
log.Warn(ctx, "Starting optimistic build failed", err)
return nil
}
// 페이로드 ID 저장
k.setOptimisticPayload(*fcr.PayloadID, nextHeight)
return nil
}
```

5. CometBFT와 EVM 엔진 상호작용

5.1 데이터 플로우

```
1. CometBFT PrepareProposal 요청
↓
2. EVM 엔진에서 새 페이로드 빌드
↓
3. 투표 확장에서 크로스체인 증명 수집
↓
4. 페이로드 + 증명을 트랜잭션으로 패키징
```

```
↓
5. CometBFT ProcessProposal 검증
↓
6. EVM 엔진에 페이로드 푸시하여 검증
↓
7. FinalizeBlock으로 상태 최종화
↓
8. PostFinalize에서 다음 블록 최적화 빌드 시작
```

5.2 합의와 실행의 분리

- **합의 레이어 (CometBFT)**: 블록 순서와 최종성 결정
- **실행 레이어 (EVM 엔진)**: 트랜잭션 실행과 상태 변경
- **브리지 레이어 (Halo)**: 두 레이어 간 통신 및 동기화

5.3 에러 처리 및 복구

```go
// 네트워크 에러에 대한 무한 재시도
err = retryForever(ctx, func(ctx context.Context) (bool, error) {
// 네트워킹 에러는 쉽게 식별할 수 없으므로 모든 에러에 대해 재시도
status, err := pushPayload(ctx, s.engineCl, payload, blockHashes)
if  err != nil {
log.Warn(ctx, "Verifying proposal failed: push new payload to evm (will retry)",
return false, nil  // 재시도
}
```

```
if  invalid, err := isInvalid(status);  invalid {
return false, errors.Wrap(err, "invalid payload, rejecting proposal") // 중단, 재
}
return true, nil  // 완료
})
```

요약

Halo의 ABCI++ 합의 플로우는 다음과 같은 특징을 가집니다:

1. **EVM 통합**: Geth 엔진과 직접 통합하여 이더리움 호환성 제공
2. **투표 확장**: 크로스체인 증명을 투표에 포함하여 합의 달성
3. **최적화된 빌드**: 다음 제안자를 예측하여 미리 페이로드 준비
4. **강력한 검증**: 다중 레벨 검증으로 보안성 확보
5. **에러 복구**: 네트워크 에러에 대한 자동 재시도 메커니즘
6. **타임아웃 관리**: 적절한 타임아웃으로 라이브니스 보장

이러한 설계를 통해 Halo는 CometBFT의 즉시 최종성과 EVM의 실행 환경을 효과적으로 결합하여 크로스체인 기능을 제공합니다.